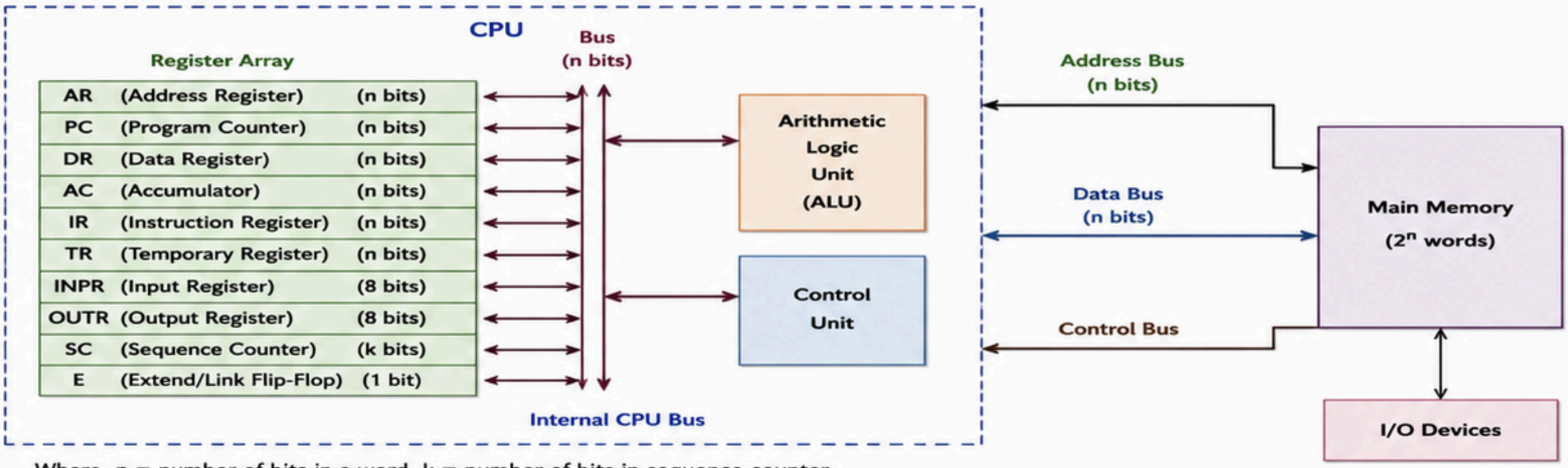
## 1. Explain the General Register Organization in a basic computer with the help of block diagram.

**Ans:**

In a basic computer, the CPU contains a set of registers that store data, addresses and control information used during processing. These registers are interconnected through internal buses and are used by the ALU, control unit and memory interface for execution of instructions.




Where, n = number of bits in a word, k = number of bits in sequence counter.

- AR holds memory address.
- PC holds address of next instruction.
- DR holds data from memory/I/O.
- AC is used for arithmetic/logic operations.
- IR holds the instruction being executed.
- TR is a temporary register.
- INPR and OUTR are used for input and output data.
- SC is used by control unit to generate timing states.
- E is carry/extend flip-flop used in arithmetic operations.

---

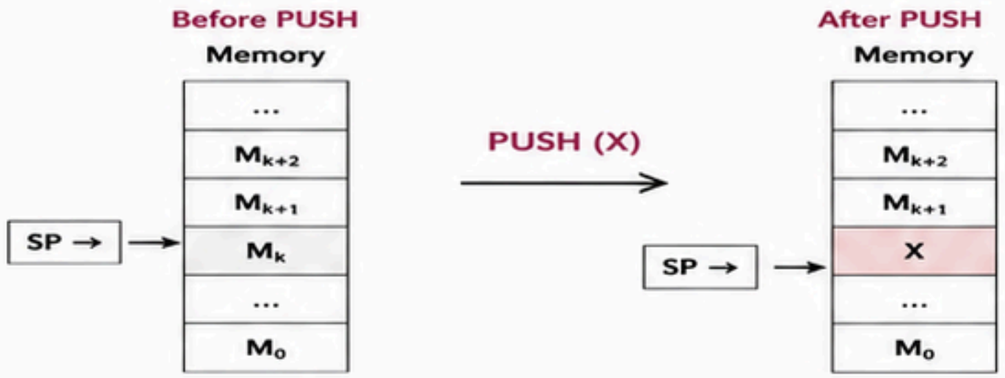
## 2. Illustrate PUSH and POP operations in stack organization.

**Ans:**

Stack is a LIFO (Last-In-First-Out) organization. Stack Pointer (SP) points to the top of the stack.

### PUSH Operation (Insert data into stack)

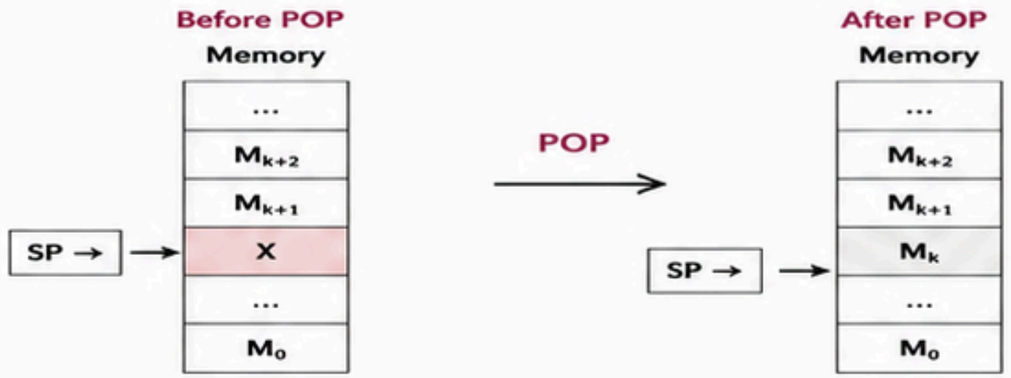
1. Decrement SP.
2. Store the data into memory location pointed by SP.
3. SP now points to the new top.




$$SP\ (new) = SP\ (old) - 1$$
$$M[SP] \leftarrow X$$

### POP Operation (Remove data from stack)

1. Read the data from memory location pointed by SP.
2. Increment SP.
3. The previous data is removed; SP points to next item.




$$Data = M[SP]$$
$$SP\ (new) = SP\ (old) + 1$$

## 3. Explain Three, Two, One and Zero addressing formats.

**Ans:** Instruction format indicates the number of explicit address fields (or operands) present in the instruction.

| Addressing Format | General Form (Fields in Instruction) | Meaning | Example Instruction | Example |
|---|---|---|---|---|
| Three Address | OPCODE \| A1 \| A2 \| A3<br>(Three address fields) | • Two source operands and one destination operand are specified explicitly.<br>• Result of operation on A1 and A2 is stored in A3. | ADD A1, A2, A3<br>A3 ← A1 + A2 | If R1 = 5, R2 = 7<br>ADD R1, R2, R3<br>⇒ R3 = R1 + R2 = 12 |
| Two Address | OPCODE \| A1 \| A2<br>(Two address fields) | • One operand is both a source and destination.<br>• Result is stored in A1. | ADD A1, A2<br>A1 ← A1 + A2 | If R1 = 5, R2 = 7<br>ADD R1, R2<br>⇒ R1 = R1 + R2 = 12 |
| One Address | OPCODE \| A1<br>(One address field) | • Only one operand is given explicitly.<br>• Other operand is implied (usually accumulator AC or top of stack).<br>• Result is stored in the implied operand. | ADD A1<br>AC ← AC + M[A1] | If AC = 5, M[100] = 7<br>ADD 100<br>⇒ AC = AC + M[100] = 12 |
| Zero Address | OPCODE<br>(No address field) | • No operand is specified.<br>• Operands are implied (usually top two items of stack).<br>• Result is stored in implied location (usually top of stack). | ADD<br>TOS ← (TOS − 1) + TOS | If stack top two items are 5 and 7<br>ADD<br>⇒ Top of stack = 5 + 7 = 12 |

Note: In zero-address instructions, operands are implied by the stack; in one-address by accumulator; in two/three-address by explicit fields.

## 4. Compare RISC and CISC architectures.

**Ans:**

| Feature | RISC (Reduced Instruction Set Computer) | CISC (Complex Instruction Set Computer) |
|---|---|---|
| 1. Instruction Set | Small and simple instruction set. | Large and complex instruction set. |
| 2. Instruction Length | Fixed length (e.g., 32 bits). | Variable length. |
| 3. Instruction Format | Few formats. | Many formats. |
| 4. Addressing Modes | Few addressing modes. | Many addressing modes. |
| 5. Memory Access | Load/Store architecture. Only LOAD/STORE access memory. | Memory-to-memory operations are allowed. |
| 6. Registers | Large number of general purpose registers. | Smaller number of general purpose registers. |
| 7. Control Unit | Hardwired control unit. | Microprogrammed control unit. |
| 8. Execution Time | Most instructions execute in one clock cycle. | Instructions may take multiple cycles. |
| 9. Pipelining | Easy to pipeline. | Difficult to pipeline. |
| 10. Code Density | Lower (more lines of code for a program). | Higher (fewer lines of code for a program). |
| 11. Performance | Higher performance due to simple instructions and pipelining. | Lower performance due to complex instructions. |
| 12. Examples | ARM, MIPS, SPARC, RISC-V, PowerPC. | x86, 8086/8088, 68000, VAX. |
| 13. Power Consumption | Lower. | Higher. |
| 14. Design Complexity | Simpler hardware design. | More complex hardware design. |

## 5. Classify instructions based on functionality and discuss data transfer and arithmetic instructions.

**Ans:** Computer instructions are classified into groups based on the operation they perform.

| Classification (by Functionality) | Description | Examples |
|---|---|---|
| 1. Data Transfer Instructions | These instructions move data from one location (source) to another (destination) without changing the data. | • MOV R1, R2 ; R1 ← R2<br>• LOAD R1, 1000H ; R1 ← M[1000H]<br>• STORE 1000H, R1 ; M[1000H] ← R1<br>• IN R1, PORT ; Input from I/O port to R1<br>• OUT PORT, R1 ; Output from R1 to I/O port |
| 2. Arithmetic Instructions | These instructions perform arithmetic operations such as addition, subtraction, increment, decrement, multiplication and division. | • ADD R1, R2 ; R1 ← R1 + R2<br>• SUB R1, R2 ; R1 ← R1 − R2<br>• INC R1 ; R1 ← R1 + 1<br>• DEC R1 ; R1 ← R1 − 1<br>• MUL R1, R2 ; R1 ← R1 × R2<br>• DIV R1, R2 ; R1 ← R1 / R2 |
| 3. Logical Instructions | These instructions perform logical operations like AND, OR, NOT, XOR, etc. | • AND R1, R2<br>• OR R1, R2<br>• NOT R1<br>• XOR R1, R2 |
| 4. Shift Instructions | These instructions shift the bits of a data item left or right. | • SHL R1 ; Logical shift left<br>• SHR R1 ; Logical shift right<br>• ROL R1 ; Rotate left<br>• ROR R1 ; Rotate right |
| 5. Program Control Instructions | These instructions change the sequence of execution of instructions. | • JMP 2000H ; Unconditional jump<br>• JZ 3000H ; Jump if zero<br>• CALL 4000H ; Subroutine call<br>• RET ; Return from subroutine |
| 6. I/O Instructions | These instructions are used to transfer data between CPU and I/O devices. | • IN R1, PORT<br>• OUT PORT, R1 |

### Data Transfer Instructions

- These instructions do not alter the content of data; they only move or copy the data.
- Addressing modes used: register, immediate, direct, indirect, indexed, etc.
- Example: MOV R1, R2 → contents of R2 are copied to R1.

### Arithmetic Instructions

- These instructions perform arithmetic operations on binary data.
- The result is stored in a register or memory location.
- Flags (Carry, Zero, Sign, Overflow, etc.) are affected according to the result.

## 6. Classify instructions based on functionality and discuss logical and shift instructions.

**Ans:** Instructions are classified by the function they perform. Here we discuss Logical and Shift instructions in detail.

### Logical Instructions

These instructions perform bit-wise logical operations on data.
They affect the condition flags (Zero, Sign, Parity, etc.).

| Instruction | Operation | Description | Example | Operation |
|---|---|---|---|---|
| AND | A ← A · B | Bit-wise AND operation. | AND R1, R2 | R1 ← R1 AND R2 |
| OR | A ← A + B | Bit-wise OR operation. | OR R1, R2 | R1 ← R1 OR R2 |
| XOR | A ← A ⊕ B | Bit-wise exclusive OR operation. | XOR R1, R2 | R1 ← R1 XOR R2 |
| NOT | A ← $\overline{A}$ | 1's complement (bit-wise complement). | NOT R1 | R1 ← $\overline{R1}$ |
| TEST | Flags set | Performs AND operation but result is not stored. | TEST R1, R2 | Flags updated |

**Example:**
R1 = 1100 1010 R2 = 1010 1100

| Operation | Result |
|---|---|
| R1 AND R2 | 1000 1000 |
| R1 OR R2 | 1110 1110 |
| R1 XOR R2 | 0110 0110 |
| NOT R1 | 0011 0101 |

**Note:**
- Logical operations are performed bit by bit.
- No carry is generated.
- Used in masking, testing and comparing data.

### Shift Instructions

These instructions shift the bits of a register or memory location either left or right. Some shifts are logical and some are arithmetic.

| Instruction | Operation | Description | Example | Effect |
|---|---|---|---|---|
| Logical Shift Left (SHL) | A ← A << 1 | Shift left; 0 enters in LSB; MSB shifts out into Carry flag. | SHL R1 | R1 ← R1 << 1<br>CF ← old MSB |
| Logical Shift Right (SHR) | A ← A >> 1 | Shift right; 0 enters in MSB; LSB shifts out into Carry flag. | SHR R1 | R1 ← R1 >> 1<br>CF ← old LSB |
| Arithmetic Shift Right (ASHR) | A ← A (arith) >> 1 | Shift right; MSB (Sign bit) is replicated. | ASHR R1 | Sign extension preserved |
| Rotate Left (ROL) | Rotate Left | MSB shifted out enters in LSB. | ROL R1 | All bits rotate left |
| Rotate Right (ROR) | Rotate Right | LSB shifted out enters in MSB. | ROR R1 | All bits rotate right |

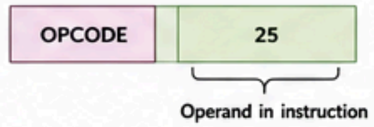
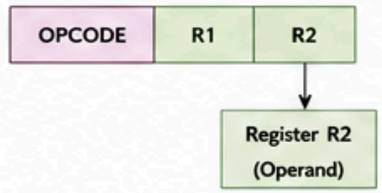
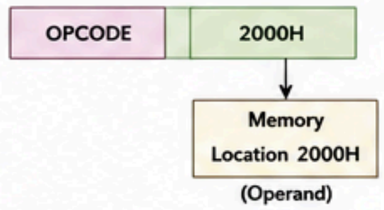
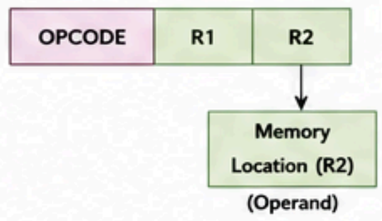
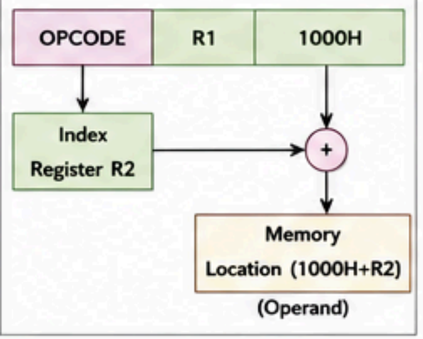
**Example:**
R1 = 1101 0110

| SHL R1 | SHR R1 | ASHR R1 (MSB=1) | ROL R1 | ROR R1 |
|---|---|---|---|---|
| 1010 1100 | 0110 1011 | 1110 1011 | 1010 1101 | 0110 1011 |

**Note:**
- Shifts are used for multiplication/division by $2^n$.
- Rotates are used for cyclic movement of bits.

## 7. Illustrate any five addressing modes.

**Ans:** Addressing mode specifies the way the operand is specified in an instruction.
Five commonly used addressing modes are as follows:

| Addressing Mode | Description | Example | Diagram |
|---|---|---|---|
| 1. Immediate Addressing | The operand is part of the instruction itself. No address is required. | ADD R1, #25 (25 is the operand) |   |
| 2. Register Addressing | The operand is in a register. The instruction specifies the register. | ADD R1, R2 (Operand in R2) |   |
| 3. Direct Addressing | The instruction contains the memory address of the operand. | ADD R1, 2000H (Operand at memory location 2000H) |   |
| 4. Register Indirect Addressing | The instruction specifies a register which contains the address of the operand. | ADD R1, (R2) (Address in R2, operand at memory[R2]) |   |
| 5. Indexed Addressing | The effective address is obtained by adding the contents of an index register to a displacement (address) given in the instruction. | ADD R1, 1000H[R2] (EA = 1000H + (R2)) |   |

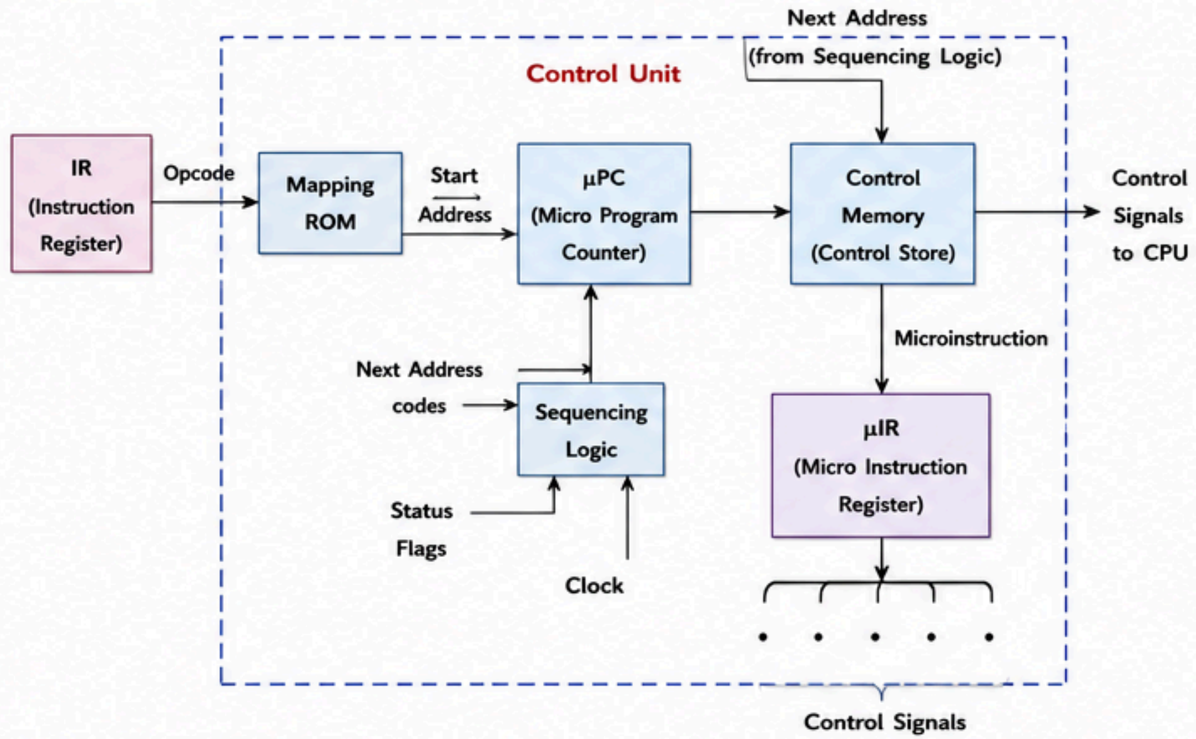
**Note:** Many other addressing modes exist such as Indirect, Base, Relative, Auto-increment, Auto-decrement, etc.

## 8. Explain Microprogrammed control organization.

**Ans:**

In microprogrammed control, control signals are generated by executing a program stored in a control memory (also called control store). Each word in this memory is a microinstruction. The sequence of microinstructions forms a control program.

**Block Diagram:**




**Working:**

1. Opcode of the instruction in IR is applied to the Mapping ROM which gives the starting address of the microprogram for that instruction.
2. This address is loaded into µPC.
3. µPC addresses the Control Memory and the microinstruction is read into µIR.
4. The microinstruction contains control signals and sequencing information.
5. Control signals are issued to the CPU to perform micro-operations.
6. Sequencing Logic, using condition codes (status flags) and sequencing field of µIR, determines the next address and loads it into µPC.
7. Steps 3–6 repeat until an END (or next instruction) microinstruction is encountered.

**Advantages:**

- Easy to design and modify (just change microprogram).
- Complex instructions are easy to implement.
- Provides a high degree of flexibility.

**Disadvantages:**

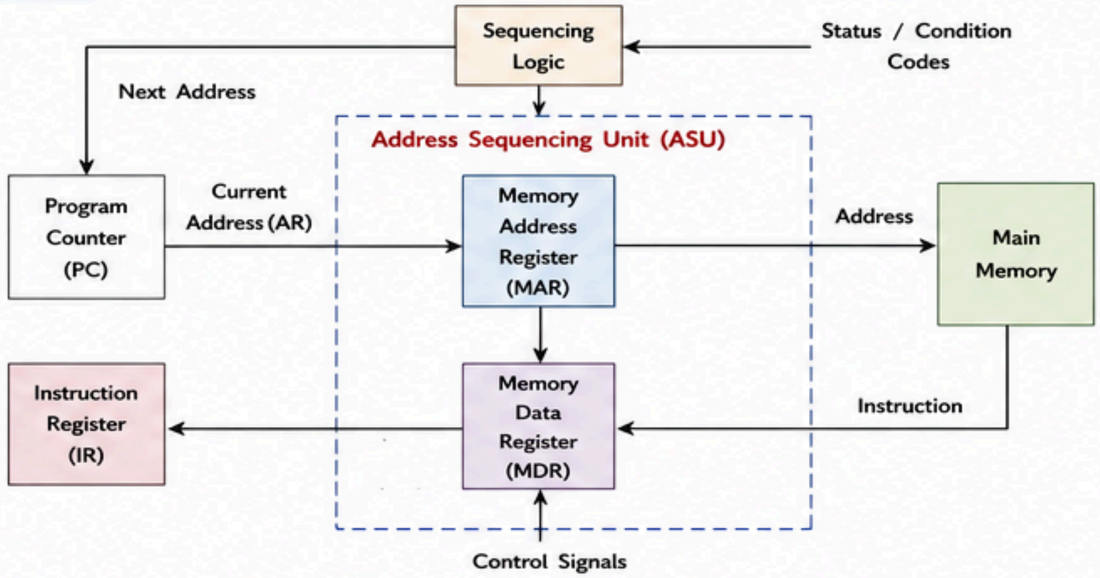
- Control memory is required (more hardware).
- Slower than hardwired control due to microinstruction fetch.

## 9. Discuss the concept of address sequencing with the help of block diagram.

**Ans:**

Address sequencing is the process of generating the addresses of successive memory locations from which instructions or data are to be fetched during program execution. The addresses are generated in a predetermined sequence.

**Block Diagram:**




**Working:**

1. The current address of the next memory location is held in the Program Counter (PC).
2. The contents of PC are transferred to MAR.
3. The address in MAR is placed on the address bus and the memory is read.
4. The instruction read from memory is stored in MDR and then transferred to IR.
5. Sequencing Logic generates the next address based on the current address, instruction type, and status/condition codes.
6. The next address is loaded into PC for the next cycle.

**Types of Address Sequencing:**

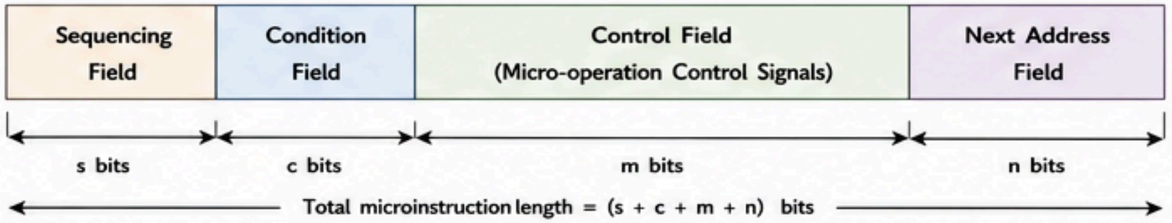
| Sequencing Mode | Description |
|---|---|
| 1. Sequential Addressing | The next address is the subsequent address. PC ← PC + 1 |
| 2. Branch Addressing | The next address is obtained from the address field of the instruction. |
| 3. Indirect Addressing | The address field points to a memory location containing the next address. |
| 4. Indexed Addressing | The next address is obtained by adding an index to a base address. |
| 5. Subroutine Addressing | The return address is saved and the address of the subroutine is loaded into PC. |
| 6. Interrupt Addressing | The address of the interrupt service routine is loaded into PC. |

**Note:** Proper address sequencing ensures correct order of instruction execution and program flow.

## 10. Explain the fields of Microinstruction format.

**Ans:**

A microinstruction specifies the micro-operations to be performed in a single clock period. The format of microinstruction depends on the control design. A typical horizontal microinstruction format is shown below.




| Field | Purpose | Contents / Function |
|---|---|---|
| 1. Sequencing Field | Provides information related to sequencing of microinstructions. | • Specifies how the next microinstruction address is to be generated.<br>• Includes control for selecting the source of next address. |
| 2. Condition Field | Specifies the condition under which the branch to the next address is to occur. | • Contains condition codes to test processor status.<br>• Examples: Z (Zero), C (Carry), N (Negative), V (Overflow), etc.<br>• Used with sequencing field to decide the next microinstruction. |
| 3. Control Field | Specifies the micro-operations to be performed during the microinstruction cycle. | • Consists of a group of control signals (one bit per signal).<br>• Each bit corresponds to a register transfer, ALU operation, memory operation, I/O operation, etc.<br>• A '1' in a bit position activates the corresponding control signal. |
| 4. Next Address Field | Specifies the address of the next microinstruction. | • Contains the address (or part of address) of the next microinstruction.<br>• It may be a fixed address or obtained from a source using sequencing information.<br>• Loaded into μPC at the end of the cycle. |

**Notes:**

- In horizontal microinstruction format, the control field is wide (one bit per control signal).
- In vertical microinstruction format, the control field is encoded (n bits select one micro-operation).
- The length and organization of fields depend on the computer design and control requirements.

**Example:**

| Field | Bits | Description |
|---|---|---|
| Sequencing Field (S) | 3 | Select next address source (sequential, branch, etc.) |
| Condition Field (C) | 2 | Test Z, C, N, V conditions |
| Control Field (M) | 20 | 20 control signals for micro-operations |
| Next Address Field (N) | 7 | Address of next microinstruction (0–127) |
| **Total Length** | **32 bits** | (3 + 2 + 20 + 7) |